Fundamentals & Architecture

Q1: What is .NET Core (now .NET)?

A: .NET Core was the cross-platform, open-source evolution of .NET. Since **.NET 5**, Microsoft unified the platform under **".NET"** (current LTS versions: .NET 6, .NET 8). It runs on Windows, Linux, macOS, supports multiple app models (web/console/desktop/cloud), and uses a modular, NuGet-based distribution model.

Q2: What are the key components of an ASP.NET Core request pipeline?

A: A pipeline of **middleware** (e.g., routing, authentication, authorization, static files). Each middleware can **inspect**, **modify**, or **short-circuit** the request, then optionally call next() for the subsequent component.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) =>
{
    // pre-processing
    await next();
    // post-processing
});

app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/", () => "Hello");
app.Run();
```

Q3: What's the difference between Framework-dependent deployment (FDD) and Self-contained deployment (SCD)?

A:

- **FDD:** App depends on a shared runtime installed on the host. Smaller binaries; use dotnet MyApp.dll.
- **SCD:** App bundles the runtime; larger size but no dependency on host-installed runtime; runs directly ./MyApp.

Dependency Injection (DI) & Service Lifetimes

Q4: How does built-in DI work in ASP.NET Core?

A: ASP.NET Core includes a DI container; register services via AddSingleton, AddScoped, AddTransient. The framework injects dependencies via constructors.

```
builder.Services.AddSingleton<ICache, MemoryCacheImpl>();
builder.Services.AddScoped<IRepository, EfRepository>();
builder
```

- **Singleton:** One instance for the app lifetime.
- **Scoped:** One instance per **HTTP request**.
- **Transient:** New instance each time requested.

Pitfall: Don't capture **scoped** services inside **singletons**.

Configuration & Options Pattern

Q5: How do you bind configuration to strongly-typed options?

A: Use IOptions<T> / IOptionsMonitor<T> / IOptionsSnapshot<T>.

```
// appsettings.json
// "Smtp": { "Host": "smtp.example", "Port": 587, "UseSsl": true }

builder.Services.Configure<SmtpOptions>(
    builder.Configuration.GetSection("Smtp"));

// Inject IOptions<SmtpOptions>
public class Mailer
{
    private readonly SmtpOptions _opt;
    public Mailer(IOptions<SmtpOptions> options) => _opt = options.Value;
}
``
```

- IOptions<T>: static after startup.
- IOptionsMonitor<T>: watches changes.

- IOptionsSnapshot<T>: per request (scoped).

Middleware vs Filters vs Handlers

Q6: Middleware vs Filters?

A:

- Middleware: Global pipeline; runs before routing typically; applies cross-cutting concerns (logging, auth).
- Filters (MVC): Run within MVC action invocation (authorization, resource, action, exception filters) and have fine-grained control at controller/action level.

Q7: When would you use DelegatingHandler in minimal APIs?
A: In HttpClient pipelines to add cross-cutting concerns (retry, auth, custom headers).

```C#
public class AuthHandler : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request, CancellationToken ct)
    {
        request.Headers.Add("X-Api-Key", "secret");
        return await base.SendAsync(request, ct);
    }
}
```

Routing & Minimal APIs

Q8: What are Minimal APIs?
A: Lightweight API endpoints defined directly in Program.cs without controllers—great for microservices and quick apps.

```
var app = WebApplication.Create();
app.MapGet("/hello", () => Results.Ok("Hi!"));
app.MapPost("/items", (Item item) => Results.Created($"/items/{item.Id}", item));
app.Run();
```

Tips: Use MapGroup, WithOpenApi, and endpoint filters for organization and documentation.

Authentication & Authorization

Q9: How does ASP.NET Core handle authentication?
A: Via **Authentication handlers** (cookies, JWT bearer, OpenID Connect) and **Authorization policies/requirements**.

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", opts =>
    {
        opts.Authority = "https://identity.example";
        opts.Audience = "api1";
        opts.RequireHttpsMetadata = true;
    });

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));
});

```

Pitfall: Ensure app.UseAuthentication() is placed **before** app.UseAuthorization().

EF Core & Data Access

Q10: What's the difference between Change Tracking and No-Tracking queries?

A:

- **Tracking:** EF tracks entity state for updates (DbContext has overhead).
- **No-Tracking:** Better for read-only queries; use AsNoTracking().

```
var products = await db.Products.AsNoTracking()
    .Where(p => p.IsActive)
    .ToListAsync();
```

Q11: How to implement soft deletes?

A: Use a IsDeleted flag and global query filters.

```
modelBuilder.Entity<Post>().HasQueryFilter(p => !p.IsDeleted);
```

Caveat: Remember filters apply everywhere; disable with IgnoreQueryFilters() when needed.

Q12: How do you handle migrations safely?

A:

- Keep migrations small and frequent.
- Use **transactions** for complex migrations.
- Avoid destructive changes on hot paths; test on staging.
- Use dotnet ef migrations add <Name> and dotnet ef database update.

Async & Performance

Q13: Why prefer async in ASP.NET Core?
A: Non-blocking I/O allows better scalability with fewer threads. Use async/await where I/O-bound operations occur (database, HTTP calls, file I/O).

```
app.MapGet("/data", async (DbContext db) =>
{
    var items = await db.Items.ToListAsync();
    return Results.Ok(items);
});
```

Q14: How to improve performance?

A:

- Enable **Response Compression** and **HTTP/2**.
- Use **Pooling** (DbContext pooling, HttpClientFactory).
- Cache frequently accessed data (MemoryCache/DistributedCache).
- Avoid synchronous blocking (.Result, .Wait()).

```
builder.Services.AddResponseCompression();
builder.Services.AddDbContextPool<AppDbContext>(opts =>
    opts.UseSqlServer(connString));
builder.Services.AddHttpClient("api")
    .SetHandlerLifetime(TimeSpan.FromMinutes(5))
    .AddPolicyHandler(retryPolicy);
``
```

Logging & Observability

Q15: How does logging work in ASP.NET Core?

A: Built-in **ILogger**, providers (Console, Debug, EventSource, Serilog, etc.), structured logging with scopes.

```
public class Worker(ILogger<Worker> logger)
{
    public void Do() => logger.LogInformation("Processing {ItemId}", 123);
}

```

Tip: Configure Log Levels via appsettings.json and use **Activity/DiagnosticSource** + OpenTelemetry for distributed tracing.

Caching

Q16: Difference between MemoryCache and DistributedCache?

A:

- **MemoryCache**: Fast, in-memory, per-instance; not shared across servers.
- **DistributedCache**: External backing (Redis/SQL); shared across instances; suitable for web farms.

```
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = "localhost:6379";
    options.InstanceName = "myapp:";
});
```

Q17: What's IHostedService / BackgroundService?

A: Long-running background tasks with lifecycle managed by the host.

```
public class MetricsCollector : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            // collect metrics
            await Task.Delay(TimeSpan.FromSeconds(10), ct);
        }
    }
}

builder.Services.AddHostedService<MetricsCollector>();
```

Pitfall: Don't run CPU-intensive work on thread pool without limits; use queues/backpressure.

gRPC vs REST

Q18: When choose gRPC over REST?

A: gRPC offers **binary**, **HTTP/2**, **contract-first** via Protocol Buffers; better for microservices/internal high-throughput scenarios. REST is more interoperable with browsers and simpler for public APIs.

Security

Q19: How do you protect against CSRF in ASP.NET Core?

A: For cookie-based auth, use **Antiforgery tokens** ([ValidateAntiForgeryToken] in MVC), ensure **SameSite**, **Secure**, and **HttpOnly** cookie flags. For **JWT APIs**, CSRF is typically **not applicable** (no cookies).

Q20: How do you secure secrets?

A: Use **User Secrets** in dev, **environment variables**, **Azure Key Vault** in prod; avoid committing secrets to source control.

```
dotnet user-secrets init
dotnet user-secrets set "Smtp:Password" "supersecret"
``
```

Testing

Q21: How to test minimal APIs?

A: Use WebApplicationFactory<TEntryPoint> from Microsoft.AspNetCore.Mvc.Testing for integration tests.

```
public class ApiTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;
    public ApiTests(WebApplicationFactory<Program> factory) =>
        _client = factory.CreateClient();

    [Fact]
    public async Task Hello_ReturnsOk()
    {
        var res = await _client.GetAsync("/hello");
        res.EnsureSuccessStatusCode();
    }
}
```

Versioning & Backwards Compatibility

Q22: How to version APIs in ASP.NET Core?

A: Use **ASP.NET API Versioning** package (Microsoft.AspNetCore.Mvc.Versioning) with attributes or URL/query/header-based versioning.

```
services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
});
``
```

Globalization & Localization

Q23: How do you enable localization?

A: Configure RequestLocalizationOptions, add IStringLocalizer, and resource files per culture.

```
builder.Services.AddLocalization();
var supportedCultures = new[] { "en-US", "fr-FR" };
app.UseRequestLocalization(new RequestLocalizationOptions()
    .SetDefaultCulture("en-US")
    .AddSupportedCultures(supportedCultures)
    .AddSupportedUICultures(supportedCultures));
```

File Uploads & Large Requests

Q24: How to handle large file uploads safely?

A:

- Set FormOptions.MultipartBodyLengthLimit.
- Stream to disk/cloud; avoid loading entire files in memory.
- Validate content type and size.

```
app.MapPost("/upload", async (HttpRequest req) =>
{
    var form = await req.ReadFormAsync();
    foreach (var file in form.Files)
    {
        using var stream = File.Create(Path.Combine("uploads", file.FileName));
        await file.CopyToAsync(stream);
    }
    return Results.Ok();
});
```

SignalR (Realtime)

Q25: What is SignalR and when to use it?

A: A library for real-time web functionality (WebSockets, Server-Sent Events fallback). Ideal for dashboards, chats, live notifications.

```
builder.Services.AddSignalR();
app.MapHub<ChatHub>("/chat");

public class ChatHub : Hub
{
    public Task Send(string user, string message) =>
        Clients.All.SendAsync("Receive", user, message);
}
```

Hosting & Deployment

Q26: How do you host ASP.NET Core behind Nginx?

A: Run Kestrel, reverse-proxy through Nginx. Enable X-Forwarded-* headers with UseForwardedHeaders. Configure health checks, graceful shutdown, and systemd service.

Common Pitfalls

Q27: Why do I get "Cannot access a disposed object: DbContext"?

A: You used DbContext outside its **scope** or stored it in a singleton. Use **scoped** lifetime and keep db operations within request lifetime.

Why does my HttpClient exhaust sockets?

A: You created new HttpClient per request. Use **HttpClientFactory** to manage handler lifetimes and connection pooling.

```
builder.Services.AddHttpClient();
``
```

Q29: How to diagnose "Method Not Allowed (405)" in ASP.NET Core?

A: Check routing/mapping and allowed HTTP methods. Ensure you've mapped the endpoint with MapGet, MapPost, etc., or MVC action attributes like [HttpPost]. Verify middleware order and endpoint URL.

Bonus: Sample Clean Architecture Structure

```
src/
  Api/           -> ASP.NET Core host (controllers/minimal APIs)
  Application/   -> Use cases (CQRS, services, validators)
  Domain/        -> Entities, value objects, domain events
  Infrastructure -> EF Core, implementations, external services
tests/
  UnitTests/
  IntegrationTests/
```

- Keep **Domain** free of infrastructure dependencies.
- Use **MediatR** for request/response patterns and vertical slice architecture if preferred.